Taller Representación de datos

1. Agregar los Breakpoints y ejecutar le programa.

- (gdb) print int_max
  $1 = 32767
- (gdb) print int_main
  $2 = -137661553
- (gdb) print uint_main
  $3 = 32767
- (gdb) print float_max
  $4 = 0
- (gdb) print float_min
  $5 = -nan (0x7fd9a8)
- (gdb) print float_lowest
  $6 = $4.591634 68 e^{-41}$

Las variables todavía no tienen valor definido.

2. Depués de agregar el Break 34

(gdb) Break 34
(gdb) continuo

Initial Values:

int_max = 2147483647	+ = 1
int_min = -2147483648	- = 1
uint_max = 42949667295	+ = 1
uint_min = 0	
float_max = 3.40282 e+38	* = 2.0f
float_min = 1.17549 e-38	/ = 2.0f
float_lowest = -3.40282 e+38	+ = 2.0f

Verificación

Enteros con signo:

$n = 32$ bits

$$Max = 2^{31} - 1 = 2147483647$$

$$Min = -2^{31} = -2147483648$$

Enteros sin signo:

$$0 \leq x \leq 2^n - 1$$

$$Max = 2^{32} - 1 = 4294967295$$

$$Min = 0$$

Punto flotante:

1	8	23
	exponente	fracción
signo		

Float_max

Signo: 0, +

Exponente: $E = e + 127 = 127 + 127 = 254 \rightarrow 1111\ 1110$

Fracción: Para q el número sea lo más grande posible.

$$\underbrace{11111111\ 11111111\ 11111111}_{1 - 2^{-23} = 0.99}$$

$$0 \quad 1111\ 1110 \quad \underbrace{11111111\ 11111111\ 11111111}_{1 - 2^{-23} = 0.99}$$

$$N = (-1)^s \times 1 + \text{fracción} \times 2^{exp - 127}, \quad 1 \leq exp \leq 254$$

$$\cancel{(-1)^0} \times 1 + (1 - 2^{-23}) \times 2^{254 - 127}$$

$$1 \times 2 - 2^{-23} \times 2^{127}$$

$$3.4028235 \times 10^{38}$$

$1 + \text{fracción} =$

$1.\text{fracción}$

(gdb) float_min

Signo = 0 → + $\quad 2^{-126} = 1.1749 \times 10^{-38}$

Exponente = 1 el más bajito sin ser 0 $\quad$ 0000 0001

Fracción = El número más pequeño. $\quad$ 00000000000000000000000

0 0000 0001 00000000000000000000000

$$N = (-1)^s \times 1 + \text{fracción} \times 2^{exp-127} \qquad 1 \leq exp \leq 254$$

$$(-1)^0 \times 1 + 0 \times 2^{1-127}$$

$$1 \times 2^{-126}$$

$$1{,}175494 \times 10^{-38}$$

Float_Lowest

Signo = 1, -

Exponente = $E = e + 127 = 127 + 127 = 254 \rightarrow$ 1111 1110

Fracción = Para q el número sea lo más grande posible.

11111111 11111111 11111111

1 1111 1110 $\underbrace{11111111 11111111}_{1-2^{-23} = 0.99}$ 11111111

$$N = (-1)^s \times 1 + \text{fracción} \times 2^{exp}$$

$$(-1) \times 1 + (1 - 2^{-23}) \times 2^{127}$$

$$-1 \times 2 - 2^{-23} \times 2^{127}$$

$$-3.1028235 \times 10^{38}$$

8. Observe contenidos de memoria

x dirección $\quad$ Muestra los bytes en memoria donde estan guardados.

& dirección de memoria

(gdb) x &int_max

0x7FFFFFFF d864 : 0x80000000

(gdb) x &int_min

0x7fffffffd868: 0x7fffffff

(gdb) x &uint_max

0x7fffffffd86c: 0x00000000

(gdb) x &float_max

0x7fffffffd874: 0x7f800000

(gdb) x &float_min

0x7fffffffd878: 0x00400000

(gdb) x &float_lowest

0x7fffffffd87c: 0xff800000

9. Mirar contenido de variable en binario

- (gdb) print /t int_max += 1

  10000000000000000000000000000000 — Overflow — resul: -2147483648

  inicio = 01111111111111111111111111111111 — Orig: 2147483647

- (gdb) print /t int_min -= 1

  11111111111111111111111111111111 — Underflow — resul: 2147483647

  inicio = 10000000000000000000000000000000 — Orig: -2147483648

- (gdb) print /t uint_max += 1

  00000000000000000000000000000000 — Wrap-around — rest: 0

  inicio = 11111111111111111111111111111111 — valor se reinicia — Orig: 4294967295

- (gdb) print /t float_max *= 2.0f;

  0 | 11111111 | 00000000000000000000000

  inicial 0 | 11111111 0 | 11111111111111111111111

inicio:

Float_max = 340282346638528859811704183484516925440

$\underbrace{111111111111111111111111}_{24\ 1's}\ \underbrace{000\ldots00}_{104\ 0's}$

$3.40282347 \times 10^{38}$

$1{,}111111111111111111111111100\ldots00 \times 2^{127}$

127 espacios la coma

0 $[-126, 127]$

$1 \leq exp \leq 254$

Signo: 0 +

Exponente = 127 + 127 = 254 1111 1110

Fracción = 11111111111111111111111

0 | 1111 1110 | 11111111111111111111111

Fracción = 0

Signo 0 → +∞

0 ||||||| 00...00

Signo 1 → −∞

1 ||||||| 00...00

Fracción ≠ 0

No es un numero válido

Despues:

Float_max = Float_max + Float_max

sumarle 1 al exponente

$(-1)^s \times 1.\text{Fracción} \times 2^{exp} \times 2$

$(-1)^s \times 1.\text{Fracción} \times 2^{exp+1}$

$(-1)^s \times 1.\text{Fracción} \times 2^{254+1}$

$(-1)^s \times 1.\text{Fracción} \times 2^{255}$

→ infinito

+∞

```
 1111 1110
         1 +
 ---------
 1111 1111
```

0 | 11111111 | 00000000000000000000000

• (gdb) print /t float_min /= 2.0f

10000000000000000000000

inicial

inicial :

128 # comple

float_min: $1.17549435 \times 10^{-38}$

0,00000000000000000000117549435

Binario = $0.\underbrace{000....00}_{126}1$,

$1.00000000000000000000000 \times 2^{-126}$

Signo = 0 +

Exponente: 127 + (-126) = 1 00000001

Fracción: 00000000000000000000000

0 00000001 00000000000000000000000

después :

Exponente 00000001
- 1
00000000

0! es un caso especial

Se mueve el 1 implicito para no perder el valor (shifter)

0 0000 0000 10000000000000000000000

1.000

↳ implicito no se guarda

- (gdb) print /t float_lowest * 2.0 f

1 | 11111111 | 00000000000000000000000

Infinito $-\infty$

float_lowest = $-3.40282347 \times 10^{38}$

$-340282346385288598117041834845169254400$

$\underbrace{111111111111111111111111}_{24\ 1's}\ \underbrace{000\ldots000}_{104\ 0's}$

$-3.40282347 \times 10^{38}$

$1.11111111111111111111111100\ldots00 \times 2^{127}$

127 espacios la coma

Signo = 1 -

Exponente = 127 + 127 = 254 1111 1110

Fracción = 11111111111111111111111

1 | 1111 1110 | 11111111111111111111111

después:

exp + 1 1111 1110
1 +

1111 1111

1 | 1111 1111 | 00000000000000000000000

---

x, ÷ solo cambia exp, fracción =

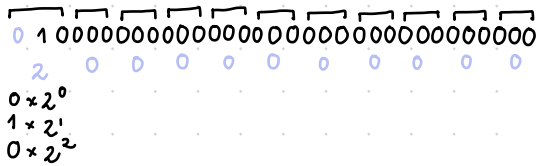
÷ y caer en denormalizado Shift Right de matisa

x # normales Se + expone y multiplican

+, - Se alinean exp y se suman

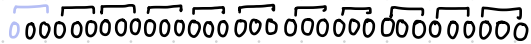
10. Ver contenido en octal

- (gdb) print /o int_max
  02000000000

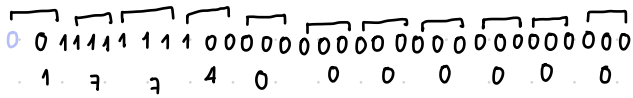


- (gdb) print /o int_min
  17777777777

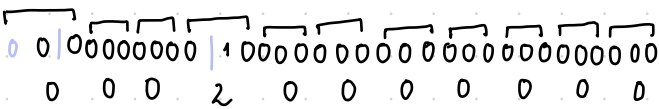
- (gdb) print /o uint_max
  00000000000



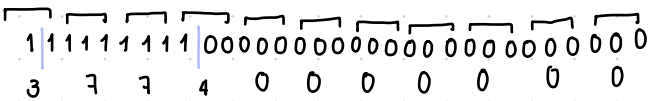
- (gdb) print /o float_max
  017740000000



- (gdb) print /o float_min
  020000000



- (gdb) print /o float_lowest
  037740000000



11. Hexadecimal

- (gdb) print /x int_max
  0x80000000

  1 0000000 0000000 0000000 0000000
  8 0 0 0 0 0 0 0

- (gdb) print /x int_min
  0x7fffffff

  0 1111111 1111111 1111111 1111111
  7 F F F F F F F

- (gdb) print /x uint_max
  0x0

  0000000 0000000 0000000 0000000
  0 0 0 0 0 0 0 0

- (gdb) print / float_max
  0x7f800000

  0 11111 111 0000000 0000000 0000000
  7 F 8 0 0 0 0 0

- (gdb) print / float_min
  0x400000

  0 00000000 1000000 0000000 0000000
  0 0 4 0 0 0 0 0

- (gdb) print / float_lowest
  0xff800000

  1 11111111 0000000 0000000 0000000
  F F 8 0 0 0 0 0

1. Rango tipo de dato:

Initial Values:

int_max = 2147483647
int_min = -2147483648
uint_max = 42949667295
uint_min = 0
float_max = 3.40282 e+38
float_min = 1.17549 e-38
float_lowest = -3.40282 e+38

Estos son los los límites. std::numeric_limits es la forma en c++ de consultar las propiedades de los tipos de datos.

2. Overflow y Underflow

Hay un desvordamiento

Overflow
resul = -2147483648
orig = 2147483647

max + 1 → min
min - 1 → max

Underflow
resul = 2147483647
orig = -2147483648

Wrap-around
valor se reinicia
rest = 0
Orig = 4294967295

Se convierte en +inf (infinito) → supera su limite

Se convierte en 0 (valores cercanos a 0)

Se convierte en -inf

3. Representación Memoria.

Complemento a 2:

32 bit, y el más significativo indica el signo

Overflow - max + 1 → min
min - 1 → max

1 - ⊖
2 - ⊕

Punto Flotante

32 bit

1 Bit signo	8 Bits exponente	23 Bits Fracción

Cuando el exponente llega a su máximo, interpreta ese patrón como "infinito" o 0.

4. Representaciones Binarias.

Binario: Realidad física del Hardware

Hexa: Es la forma más práctica para el ser Humano.
4 bits binario → 1 dígito.

Octa: Agrupa de a 3 bit

Aunque el computador trabaje en binario, es importante y más práctico leer octal o hexadecimal.

Ayudan a la depurar, detectar overflow y entender la aritmética.